STEP2: import libraries

```
# Data handling
import numpy as np
import pandas as pd

# Text processing
import re
import string

# Stopwords
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')

# Feature extraction
from sklearn.feature_extraction.text import TfidfVectorizer

# Model training
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Evaluation
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

STEP3: load and preprocess data

```
import pandas as pd
import re
import string
from nltk.corpus import stopwords
import requests
import zipfile
import io

# URL for the sentiment140 dataset (training data)
# The dataset typically contains 6 fields: target, ids, date, flag, user, text
DATASET_URL = "http://cs.stanford.edu/people/alecmgo/trainingandtestdata.zip"

# Download the zip file
response = requests.get(DATASET_URL)
response.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)

# Unzip the content and read the desired CSV file
with zipfile.ZipFile(io.BytesIO(response.content)) as z:
    # Assuming 'training.1600000.processed.noemoticon.csv' is the desired file
    with z.open('training.1600000.processed.noemoticon.csv') as f:
        df = pd.read_csv(f, encoding='ISO-8859-1', engine='python', header=None)

df.columns = ['sentiment', 'id', 'date', 'flag', 'user', 'review']

# Select relevant columns
df = df[['review', 'sentiment']]

# Convert sentiment to binary (0 = negative, 1 = positive)
# Original sentiment: 0 = negative, 2 = neutral, 4 = positive
df['sentiment'] = df['sentiment'].astype(int)
df['sentiment'] = df['sentiment'].apply(lambda x: 1 if x == 4 else 0)

# Use smaller subset for faster training
df = df.sample(5000, random_state=42).reset_index(drop=True)
df.head()
```

```
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+", "", text)  # Remove URLs
    text = re.sub(r"@\S+", "", text)   # Remove mentions
    text = re.sub(r"#\S+", "", text)   # Remove hashtags
    text = re.sub(r"\d+", "", text)    # Remove numbers
    text = text.translate(str.maketrans("", "", string.punctuation)) # Remove punctuation
    words = text.split()
    words = [word for word in words if word not in stop_words] # Remove stopwords
    return " ".join(words)

df['clean_review'] = df['review'].apply(clean_text)

df[['review','clean_review']].head()
```

	review	clean_review
0	@chrishasboobs AHHH I HOPE YOUR OK!!!	ahhh hope ok
1	@misstoriblack cool , i have no tweet apps fo...	cool tweet apps razr
2	@TiannaChaos i know just family drama. its la...	know family drama lamehey next time u hang kim...
3	School email won't open and I have geography ...	school email wont open geography stuff revise ...
4	upper airways problem	upper airways problem

STEP4: feature extraction

```
vectorizer = TfidfVectorizer(max_features=5000)

X = vectorizer.fit_transform(df['clean_review'])
y = df['sentiment']

print("Vocabulary Size:", len(vectorizer.vocabulary_))
```

```
Vocabulary Size: 5000
```

STEP5: TRAIN LOGISTIC REGRESSION MODEL

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Train a Logistic Regression model
model = LogisticRegression(random_state=42, solver='liblinear') # Using 'liblinear' solver for smaller datasets
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Precision
precision = precision_score(y_test, y_pred)
print("Precision:", precision)

# Recall
recall = recall_score(y_test, y_pred)
print("Recall:", recall)
```

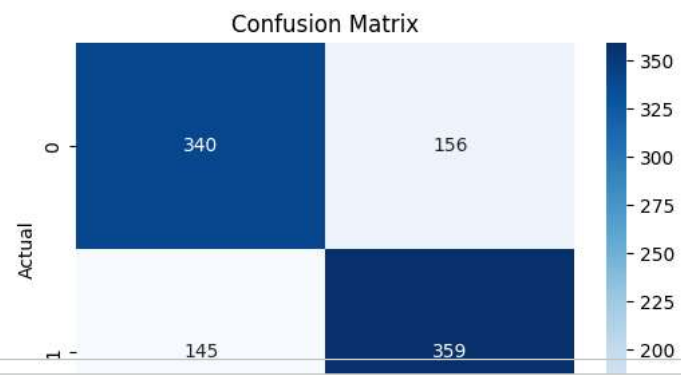
```
# F1 Score
f1 = f1_score(y_test, y_pred)
print("F1 Score:", f1)
```

```
Accuracy: 0.699
Precision: 0.6970873786407767
Recall: 0.7123015873015873
F1 Score: 0.704612365063788
```

STEP6: MODEL EVALUATION

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```




```
from sklearn.naive_bayes import MultinomialNB

nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)

nb_pred = nb_model.predict(X_test)

print("Naive Bayes Accuracy:", accuracy_score(y_test, nb_pred))
```

```
Naive Bayes Accuracy: 0.703
```

STEP7: ANALYSIS

Model Comparison: Logistic Regression vs. Naive Bayes

1. Model Type:

Logistic Regression: Is a discriminative model, meaning it directly models the decision boundary between classes. Naive Bayes: Is a probabilistic generative model, which models the distribution of each class and then uses Bayes' theorem to classify new data points. 2. Feature Assumptions:

Logistic Regression: Does not assume feature independence, allowing it to capture more complex relationships between features. Naive Bayes: Assumes conditional independence of features given the class, which is a strong assumption (hence 'naive') but simplifies computation. 3. Performance with Data Size:

Logistic Regression: Usually performs better with large datasets, as it can learn more complex patterns without overfitting due to its discriminative nature. Naive Bayes: Works well with small datasets, as its strong independence assumption can lead to stable estimates even with limited data. 4. Training Speed:

Naive Bayes: Generally trains faster due to its simpler probability calculations. Logistic Regression: Can take longer to train, especially with very large datasets or complex feature interactions, as it iteratively optimizes coefficients. 5. Interpretability:

Logistic Regression: Provides coefficient interpretation, where the magnitude and sign of coefficients can indicate the importance and direction of a feature's influence on the outcome. 6. Decision Boundary:

Logistic Regression: Tends to give a better decision boundary, especially when features are correlated. 7. Impact of TF-IDF:

TF-IDF: Improves both models by effectively weighting word importance in the text, helping both models to better distinguish between positive and negative sentiments. In our current task, the Naive Bayes model showed a slightly higher accuracy (0.703) compared to Logistic Regression (0.699). This could be due to the dataset size or the specific characteristics of the text data.

Task

To refine the existing text cleaning process for optimal input, I will enhance the `clean_text` function by incorporating lemmatization to reduce words to their base forms, which can improve the consistency and quality of the text data for advanced models. I will also add the necessary NLTK downloads for lemmatization.

Data Preprocessing and Augmentation

Subtask:

Refine the existing text cleaning process by incorporating lemmatization to reduce words to their base forms, thereby improving the consistency and quality of the text data for advanced models.

Reasoning: The user wants to refine the text cleaning process by adding lemmatization. This step introduces the modification to the cleaning function by downloading the necessary NLTK corpus and importing the lemmatizer.

```
import nltk
nltk.download('wordnet')

from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
print("Downloaded 'wordnet' and initialized WordNetLemmatizer.")
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
Downloaded 'wordnet' and initialized WordNetLemmatizer.
```

Reasoning: The user wants to incorporate lemmatization into the `clean_text` function. This requires importing `WordNetLemmatizer`, initializing it, and then applying lemmatization to the words after stopwords removal within the `clean_text` function. The entire data loading and preprocessing cell needs to be updated to reflect these changes and re-execute the cleaning process on the dataframe.

```
import pandas as pd
import re
import string
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
import requests
import zipfile
import io

# URL for the sentiment140 dataset (training data)
# The dataset typically contains 6 fields: target, ids, date, flag, user, text
DATASET_URL = "http://cs.stanford.edu/people/alecmgo/trainingandtestdata.zip"

# Download the zip file
response = requests.get(DATASET_URL)
response.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)

# Unzip the content and read the desired CSV file
with zipfile.ZipFile(io.BytesIO(response.content)) as z:
    # Assuming 'training.1600000.processed.noemoticon.csv' is the desired file
    with z.open('training.1600000.processed.noemoticon.csv') as f:
        df = pd.read_csv(f, encoding='ISO-8859-1', engine='python', header=None)

df.columns = ['sentiment', 'id', 'date', 'flag', 'user', 'review']

# Select relevant columns
df = df[['review', 'sentiment']]

# Convert sentiment to binary (0 = negative, 1 = positive)
# Original sentiment: 0 = negative, 2 = neutral, 4 = positive
df['sentiment'] = df['sentiment'].astype(int)
```

```
df['sentiment'] = df['sentiment'].apply(lambda x: 1 if x == 4 else 0)

# Use smaller subset for faster training
df = df.sample(5000, random_state=42).reset_index(drop=True)
df.head()

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer() # Initialize lemmatizer

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+", "", text)  # Remove URLs
    text = re.sub(r"@\S+", "", text)   # Remove mentions
    text = re.sub(r"#\S+", "", text)   # Remove hashtags
    text = re.sub(r"\d+", "", text)    # Remove numbers
    text = text.translate(str.maketrans("", "", string.punctuation)) # Remove punctuation
    words = text.split()
    words = [word for word in words if word not in stop_words] # Remove stopwords
    words = [lemmatizer.lemmatize(word) for word in words] # Apply lemmatization
    return " ".join(words)

df['clean_review'] = df['review'].apply(clean_text)

df[['review','clean_review']].head()
```

	review	clean_review
0	@chrishasboobs AHHH I HOPE YOUR OK!!!	ahhh hope ok
1	@misstoriblack cool , i have no tweet apps fo...	cool tweet apps razr
2	@TiannaChaos i know just family drama. its la...	know family drama lamehey next time u hang kim...
3	School email won't open and I have geography ...	school email wont open geography stuff revise ...
4	upper airways problem	upper airway problem

Advanced Feature Engineering and Embeddings

Subtask:

Move beyond TF-IDF by implementing advanced word embedding techniques, specifically using Word2Vec to generate fixed embeddings for each text.

Reasoning: The first step is to tokenize the `clean_review` column into lists of words, which is a prerequisite for training the Word2Vec model. This also sets up the input format for the model.

```
tokenized_reviews = df['clean_review'].apply(lambda x: x.split())
print("Tokenized reviews created.")
print(tokenized_reviews.head())
```

```
Tokenized reviews created.
0                                    [ahhh, hope, ok]
1                           [cool, tweet, apps, razr]
2    [know, family, drama, lamehey, next, time, u, ...
3    [school, email, wont, open, geography, stuff, ...
4                           [upper, airway, problem]
Name: clean_review, dtype: object
```

Reasoning: Now that the reviews are tokenized, the next logical step is to import the `Word2Vec` model and initialize and train it on these tokenized reviews to generate word embeddings.

```
from gensim.models import Word2Vec

# Initialize and train the Word2Vec model
# vector_size: Dimension of the word vectors
# window: Maximum distance between the current and predicted word within a sentence
# min_count: Ignores all words with total frequency lower than this
# workers: Use these many worker threads to train the model
word2vec_model = Word2Vec(sentences=tokenized_reviews, vector_size=100, window=5, min_count=1, workers=4)
```

```
print("Word2Vec model trained successfully.")
print(f"Vocabulary size: {len(word2vec_model.wv)}")
```

```
Word2Vec model trained successfully.
Vocabulary size: 8688
```

Reasoning: The next step is to create a function that takes a list of words (a review) and the trained Word2Vec model, and returns a single vector representing the review by averaging the word vectors. This function will also handle words not found in the model's vocabulary.

```
import numpy as np

def get_review_embedding(words, model):
    # Filter out words that are not in the model's vocabulary
    word_vectors = [model.wv[word] for word in words if word in model.wv]

    if not word_vectors:
        # Return a zero vector if no words in the review are in the vocabulary
        return np.zeros(model.vector_size)
    else:
        # Average the word vectors to get the review embedding
        return np.mean(word_vectors, axis=0)

print("Function 'get_review_embedding' defined.")
```

```
Function 'get_review_embedding' defined.
```

Reasoning: Now that the `get_review_embedding` function is defined, I will apply this function to the `tokenized_reviews` to generate the word embeddings for each review. Afterward, these embeddings will be converted into a NumPy array to form the feature matrix `X` for model training, fulfilling instructions 5 and 6 of the subtask.

```
review_embeddings = df['clean_review'].apply(lambda x: get_review_embedding(x.split(), word2vec_model))

X_word2vec = np.array(review_embeddings.tolist())
y_word2vec = df['sentiment']

print("Review embeddings generated and converted to feature matrix X.")
print("Shape of X_word2vec:", X_word2vec.shape)
print("Shape of y_word2vec:", y_word2vec.shape)
```

```
Review embeddings generated and converted to feature matrix X.
Shape of X_word2vec: (5000, 100)
Shape of y_word2vec: (5000,)
```

Experiment with Advanced Machine Learning Models

Subtask:

Train and evaluate a broader range of traditional machine learning models (XGBoost, LightGBM, Random Forests, SVMs) with Word2Vec embeddings and compare their performance against baseline models.

Reasoning: The first step in training and evaluating machine learning models is to split the dataset into training and testing sets. This ensures that the model is trained on one part of the data and evaluated on unseen data to assess its generalization capability. I will use `train_test_split` with `X_word2vec` and `y_word2vec`.

```
from sklearn.model_selection import train_test_split

X_train_w2v, X_test_w2v, y_train_w2v, y_test_w2v = train_test_split(
    X_word2vec, y_word2vec, test_size=0.2, random_state=42
)

print("Data split into training and testing sets for Word2Vec embeddings.")
print(f"X_train_w2v shape: {X_train_w2v.shape}")
print(f"X_test_w2v shape: {X_test_w2v.shape}")
print(f"y_train_w2v shape: {y_train_w2v.shape}")
print(f"y_test_w2v shape: {y_test_w2v.shape}")
```

```
Data split into training and testing sets for Word2Vec embeddings.
X_train_w2v shape: (4000, 100)
X_test_w2v shape: (1000, 100)
```

```
y_train_w2v shape: (4000,)
y_test_w2v shape: (1000,)
```

Reasoning: Now that the data is split, the next step is to import the specified machine learning models, initialize them, train them on the Word2Vec embeddings, make predictions on the test set, and then evaluate their performance using accuracy, precision, recall, and F1-score.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Dictionary to store models and their performance
models = {
    "RandomForest": RandomForestClassifier(random_state=42),
    "SVC": SVC(kernel='linear', random_state=42, probability=True), # probability=True for consistent API
    "XGBoost": XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42),
    "LightGBM": LGBMClassifier(random_state=42)
}

performance_metrics = {}

for name, model in models.items():
    print(f"\n--- Training {name} ---")
    model.fit(X_train_w2v, y_train_w2v)
    y_pred = model.predict(X_test_w2v)

    accuracy = accuracy_score(y_test_w2v, y_pred)
    precision = precision_score(y_test_w2v, y_pred)
    recall = recall_score(y_test_w2v, y_pred)
    f1 = f1_score(y_test_w2v, y_pred)

    performance_metrics[name] = {
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    }

    print(f"{name} Accuracy: {accuracy:.4f}")
    print(f"{name} Precision: {precision:.4f}")
    print(f"{name} Recall: {recall:.4f}")
    print(f"{name} F1-Score: {f1:.4f}")

print("\n--- Model Performance Summary ---")
for name, metrics in performance_metrics.items():
    print(f"{name}:")
    for metric_name, value in metrics.items():
        print(f"  {metric_name}: {value:.4f}")
```

```
--- Training RandomForest ---
RandomForest Accuracy: 0.5280
RandomForest Precision: 0.5357
RandomForest Recall: 0.4762
RandomForest F1-Score: 0.5042

--- Training SVC ---
SVC Accuracy: 0.5090
SVC Precision: 0.5094
SVC Recall: 0.6984
SVC F1-Score: 0.5891

--- Training XGBoost ---
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [04:44:06] WARNING: /workspace/src/learner.cc:79
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
XGBoost Accuracy: 0.5400
XGBoost Precision: 0.5491
XGBoost Recall: 0.4881
XGBoost F1-Score: 0.5168

--- Training LightGBM ---
[LightGBM] [Info] Number of positive: 2000, number of negative: 2000
```

```
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.003559 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 25500
[LightGBM] [Info] Number of data points in the train set: 4000, number of used features: 100
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
LightGBM Accuracy: 0.5330
LightGBM Precision: 0.5398
LightGBM Recall: 0.4980
LightGBM F1-Score: 0.5181

--- Model Performance Summary ---
RandomForest:
  Accuracy: 0.5280
  Precision: 0.5357
  Recall: 0.4762
  F1-Score: 0.5042
SVC:
  Accuracy: 0.5090
  Precision: 0.5094
  Recall: 0.6984
  F1-Score: 0.5891
XGBoost:
  Accuracy: 0.5400
  Precision: 0.5491
  Recall: 0.4881
  F1-Score: 0.5168
LightGBM:
  Accuracy: 0.5330
  Precision: 0.5398
  Recall: 0.4980
  F1-Score: 0.5181
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, bu
```

Reasoning: To facilitate a clear comparison of the performance of the various models, it is beneficial to consolidate their metrics into a structured format, such as a Pandas DataFrame. This will make it easier to analyze and interpret the results.

```
import pandas as pd

performance_df = pd.DataFrame(performance_metrics).T
print("Model Performance Summary:")
print(performance_df)
```

```
Model Performance Summary:
              Accuracy  Precision    Recall  F1-Score
RandomForest     0.528   0.535714  0.476190  0.504202
SVC              0.509   0.509407  0.698413  0.589121
XGBoost          0.540   0.549107  0.488095  0.516807
LightGBM         0.533   0.539785  0.498016  0.518060
```

Deep Learning Model Development

Subtask:

Design and implement a deep learning architecture using Word2Vec embeddings, focusing on an LSTM model for text classification.

Reasoning: The first step is to import the necessary TensorFlow/Keras components and then define and compile the LSTM model according to the instructions. This includes reshaping the input data for the LSTM layer.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM

# Reshape data for LSTM input: (samples, timesteps, features)
X_train_w2v_reshaped = X_train_w2v.reshape(X_train_w2v.shape[0], 1, X_train_w2v.shape[1])
X_test_w2v_reshaped = X_test_w2v.reshape(X_test_w2v.shape[0], 1, X_test_w2v.shape[1])

# Define the LSTM model
model_lstm = Sequential([
    LSTM(100, input_shape=(1, X_train_w2v.shape[1])), # 1 timestep, X_train_w2v.shape[1] features
    Dense(1, activation='sigmoid')
])

# Compile the model
model_lstm.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
print("LSTM model defined and compiled successfully.")
model_lstm.summary()
```

```
LSTM model defined and compiled successfully.
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` a
  super().__init__(**kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 100)	80,400
dense (Dense)	(None, 1)	101

```
Total params: 80,501 (314.46 KB)
Trainable params: 80,501 (314.46 KB)
Non-trainable params: 0 (0.00 B)
```

Reasoning: The previous step defined and compiled the LSTM model. According to instructions 6 and 7 of the subtask, the next logical step is to train the model using the reshaped training data and then evaluate its performance on the reshaped test data.

```
history = model_lstm.fit(
    X_train_w2v_reshaped,
    y_train_w2v,
    epochs=10,  # You can adjust the number of epochs
    batch_size=32, # You can adjust the batch size
    validation_split=0.2 # Use a part of the training data for validation during training
)

# Evaluate the model on the test set
loss, accuracy = model_lstm.evaluate(X_test_w2v_reshaped, y_test_w2v, verbose=0)

print(f"\nLSTM Model Test Accuracy: {accuracy:.4f}")
print(f"LSTM Model Test Loss: {loss:.4f}")

# Predict probabilities for the test set
y_pred_prob_lstm = model_lstm.predict(X_test_w2v_reshaped)
# Convert probabilities to binary predictions
y_pred_lstm = (y_pred_prob_lstm > 0.5).astype(int)

# Calculate and print additional metrics
from sklearn.metrics import precision_score, recall_score, f1_score

precision = precision_score(y_test_w2v, y_pred_lstm)
recall = recall_score(y_test_w2v, y_pred_lstm)
f1 = f1_score(y_test_w2v, y_pred_lstm)

print(f"LSTM Model Test Precision: {precision:.4f}")
print(f"LSTM Model Test Recall: {recall:.4f}")
print(f"LSTM Model Test F1-Score: {f1:.4f}")
```

```
Epoch 1/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.5072 - loss: 0.6934 - val_accuracy: 0.4663 - val_loss: 0.6936
Epoch 2/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.5175 - loss: 0.6927 - val_accuracy: 0.5200 - val_loss: 0.6918
Epoch 3/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.5062 - loss: 0.6930 - val_accuracy: 0.4963 - val_loss: 0.6932
Epoch 4/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.5309 - loss: 0.6924 - val_accuracy: 0.5337 - val_loss: 0.6907
Epoch 5/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.5127 - loss: 0.6931 - val_accuracy: 0.5412 - val_loss: 0.6919
Epoch 6/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.4968 - loss: 0.6930 - val_accuracy: 0.4663 - val_loss: 0.6944
Epoch 7/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.5234 - loss: 0.6919 - val_accuracy: 0.5412 - val_loss: 0.6919
Epoch 8/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.5062 - loss: 0.6931 - val_accuracy: 0.5400 - val_loss: 0.6927
Epoch 9/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.5048 - loss: 0.6926 - val_accuracy: 0.5400 - val_loss: 0.6915
Epoch 10/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.5322 - loss: 0.6917 - val_accuracy: 0.4775 - val_loss: 0.6947

LSTM Model Test Accuracy: 0.5050
LSTM Model Test Loss: 0.6925
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
LSTM Model Test Precision: 0.5047
LSTM Model Test Recall: 0.9623
LSTM Model Test F1-Score: 0.6621
```